EXAM SUMMARY · EXAM CODE CCAR-P

Claude Certified Architect – Professional: Exam Cheatsheet

Companion to the O'Reilly exam-prep course. Exam facts follow the official Exam Guide v1.0 (July 2026). Platform facts checked against Anthropic docs, October 2026.

63 items
Multiple-choice and multiple-response. Each item states how many to pick.

120 min
About 1 min 54 s per item.

720 / 1,000
Scaled cut score on 100–1,000. Not 72% raw.

$175 USD
Per attempt. Partner-tier discounts may apply.

12 months
Valid 12 months. On-time renewal is free and non-proctored.

PREREQUISITES
None. Architect – Foundations is not required.

RECOMMENDED
3+ yrs architecture; 6+ months Claude or LLMs in production.

DELIVERY
Pearson VUE, online proctored or test center. Closed book, NDA.

RETAKES
Wait 14 / 30 / 90 days. Max 4 attempts per rolling 12 months.

RESCHEDULE
Use ≥ **48 h**. The guide says 24 h; FAQ and policies say 48 h.

MQC lens. An experienced practitioner who designs, implements, and governs Claude systems in production and weighs every choice on **cost, latency, performance, safety, and maintainability**. Registration requires a Claude Partner Network company email.

BLUEPRINT: 7 DOMAINS, 38 OBJECTIVES

#	Domain	Weight	~Items	Lesson
D1	Solution Design & Architecture	17%	11	01
D2	Claude Models, Prompting & Context Engineering	13%	8	02
D3	**Integration**	**19%**	12	03
D4	Evaluation, Testing & Optimization	16%	10	04
D5	Governance, Safety & Risk Management	14%	9	05
D6	Stakeholder Communication & Lifecycle	14%	9	06
D7	Developer Productivity & Operational Enablement	7%	4	07

Counts are weight × 63, rounded. **D1 + D3 + D4 = 52%.** D5–D7 (35%) have no Architect – Foundations counterpart. Objectives name no API parameters: the exam tests decisions.

CORE EXAM LOGIC

1. **Simplest pattern that works.** Workflow → agent → multi-agent, only on evidence.
2. **Remove capability before you guard it.** Least privilege beats logging.
3. **Enforce in code, not the prompt.** Permissions, hooks, schemas, validators.
4. **Static first, then cache.** Batch what can wait.
5. **Diagnose from what changed.** Match the trigger to the layer.
6. **Evals gate every change.** Measured criteria, cost per task.
7. **Design for the handoff.** Cost, risk, reversal; runbooks, owners.

How items are written: a production scenario → a **trigger** in the stem ("only ever need", "after a document refresh", "same prompt every request") → "best reduces risk" / "most directly addresses" / "first place to investigate" → four options. **Pick the structural fix that matches the trigger and keeps the needed capability.**

THE SIX DISTRACTOR PATTERNS: NAME THE PATTERN, CROSS IT OUT

Pattern	Sounds like	Why it loses
Bigger model	"Switch to a larger model that follows instructions better."	Capability isn't the problem: authz, retrieval, or context is
Watch, don't remove	"Add logging." "Add a confirmation step."	Detective/compensating control where a preventive one exists
Prompt as policy	"Add to the system prompt that it must never..."	Instructions are best effort; rules that must hold live in code
Lossy shortcut	"Truncate to 1,000 tokens." "Smallest model regardless of fit."	Saves cost by discarding what the task needs
Wrong trigger	"Weights changed." "Temperature too low." "Window shrank."	The event in the stem wouldn't cause it
Over-engineering	"Split into five specialized agents." "Fine-tune."	Complexity without evidence; ~15× tokens for multi-agent

OFFICIAL SAMPLES (EXAM GUIDE, SECTION 8)

Sample	Trigger	Answer	Wrong options
1 · D3 Support agent can refund and delete	"Staff only ever need to read and draft"	**Remove the tools**	Logging, confirmation (watch, don't remove); bigger model
2 · D2 8,000-token prompt on every request	"Latency and cost are both concerns"	**Static first + prompt caching**	Truncate, smallest model (lossy); few-shot block (no cache)
3 · D4 RAG wrong after document refresh	"Model version unchanged"	**Investigate retrieval/indexing**	Weights, temperature, window (wrong trigger)

D1 Solution Design & Architecture

17% · ~11 items · Lesson 01

OBJECTIVES

1. Translate business problems into Claude-based AI solutions
2. Design end-to-end architectures (input → processing → output → feedback loops)
3. Select patterns (workflow, agentic, augmented LLM)
4. Design multi-agent systems and orchestration
5. Apply decomposition techniques
6. Align to value pillars (efficiency, transformation, productivity, cost, performance SLAs)

FOUR DECISIONS BEFORE CODE, IN ORDER

1. **Ownership:** Claude vs. existing systems vs. humans
2. **Shape:** augment (live, seconds) · automate (pipeline) · agent (open-ended)
3. **Reference architecture:** name the pattern
4. **Entry point:** model, context strategy, surface, route

Lens: systems own exact figures, live facts, and state · code owns control flow · humans own consequential decisions.

THE LADDER: CLIMB ONLY ON EVIDENCE

Rung	Definition	Cost
Augmented LLM	One call + retrieval, tools, memory	Least flexible
Workflow	Predefined code paths	No adaptation
Agent	LLM directs its own steps in a loop	Latency, tokens, compounding errors
Multi-agent	Orchestrator + subagents	~15× chat tokens (agents ~4×)

Flowchart drawable in advance → workflow. Agents need ground truth per step, stop conditions, sandbox.

FIVE WORKFLOWS

Pattern	Use when
Prompt chaining	Fixed sequential steps; gates between
Routing	Distinct input categories (easy → small model)
Parallelization	Sectioning (independent parts) or voting
Orchestrator-workers	Subtasks unknown until input seen
Evaluator-optimizer	Clear rubric; iteration helps

MULTI-AGENT RULES

- Beat single agent by **90.2%** on Anthropic's research eval
- **Fits:** breadth-first, parallel, info exceeds one context
- **Fails:** tightly coupled work (most coding), shared context, real-time coordination
- **Subagents don't inherit context.** Pass task, constraints, format
- Scope tools per subagent; clear boundaries; orchestrator synthesizes

DECOMPOSITION AND REFERENCE ARCHITECTURES

- **Pipeline** (stages) · **map-reduce** (partitions) · **local + integration pass** (big reviews) · **specialist** · **plan-then-execute**
- Docs Q&A → **RAG** · live state → **tool call to system of record** · bulk extraction → **batch pipeline + structured outputs** · support with actions → **least-privilege agent + approvals** · broad research → **orchestrator-workers**

END-TO-END AND VALUE

- Input (screening) → processing (pattern, model, context) → output (validation) → **feedback loop** (corrections → eval set → next version)
- No feedback loop = can't improve or detect drift
- Pillars: efficiency · transformation · productivity · cost · performance SLAs. Name the pillar **and its metric** first
- **Traps: agent for known steps · multi-agent for coupled work · RAG for live data**

D2 Models, Prompting & Context Engineering

13% · ~8 items · Lesson 02

OBJECTIVES

1. Select Claude models based on trade-offs
2. Design system prompts, templates, guardrails
3. Apply zero-shot, few-shot, chain-of-thought
4. Optimize context windows and token usage
5. Prompt reuse: caching, modular prompts, Skills

MODEL CHOICE (LINEUP AS OF OCT 2026)

Model	Fit	$/MTok in · out
Fable 5.1	Hardest reasoning, long-horizon agents	10 · 50
Opus 5.5	Agentic coding, knowledge work; "start here"	4 · 20
Sonnet 5.5	Speed + intelligence	2 · 10
Haiku 5.5	Classify, extract, route at volume	0.10 · 0.50

Decide on **your eval set**. **Tune effort before switching models.** Route by difficulty. Compare **cost per completed task**.

PROMPT CACHING

- Order **tools** → **system** → **messages**, static → dynamic. Exact-prefix match
- Up to **4 breakpoints**; min **512 tokens** on current models
- TTL 5 min (write **1.25×**) or 1 h (write **2×**); reads a fraction of base input
- Cache reads don't count toward ITPM on most models
- Killers: timestamp/user ID at the top, changed tools, changing effort

TECHNIQUES AND GUARDRAILS

- **Few-shot:** 3–5 diverse examples in tags for exact format/mapping
- **CoT:** higher effort or reasoning before the answer; arithmetic to a tool
- **Shape guarantee:** structured outputs / strict tool use. Prefill and forced `tool_choice` error on new models
- System prompt = role, scope, tone, "say I don't know." Not: hard limits, authz, secrets

CONTEXT ENGINEERING

- Smallest set of high-signal tokens; 1M window ≠ use it all
- Long doc: document first, question last, quotes before answer
- Long sessions: context editing, compaction, memory tool, facts block
- KB < ~200k tokens: all in context + cache
- **Skills:** ~100 tokens metadata each; body and files load on demand

D3 Integration

19%, the largest domain · ~12 items · Lesson 03

OBJECTIVES

1. Evaluate tool/agent config for capability bloat
2. Analyze authn/authz to find security gaps
3. Evaluate accuracy-latency trade-offs
4. Observability and monitoring at scale
5. Design RAG chunking and indexing
6. Match retrieval to data shape and query
7. Choose MCP vs. API/CLI vs. agent-to-agent
8. Progressive discovery vs. monolithic context

CAPABILITY BLOAT AND IDENTITY GAPS

Finding	Fix
50+ tools, wrong picks	Scoped sub-agents · consolidate · tool search (`defer_loading`)
Tools the role never uses	**Remove them**
Shared admin service account	Per-user delegated OAuth tokens
Identity from model text	Identity from SSO; authorize server-side
MCP token passthrough	Validate tokens issued for the server
Over-privileged creds	Scope to exposed operations
Key in browser/prompt	Backend proxy
User-supplied `file_id`	Never accept; workspace per tenant
JIT provisioning only	SCIM (deprovisions on IdP removal)

RAG PIPELINE

Stage	Decision
Before RAG	< ~200k tokens (~500 pages): long context + caching
Chunk	Structure-aware; tables and procedures whole
Contextualize	50–100 tokens of context per chunk: failures −35% / −49% (+BM25) / −67% (+rerank)
Index	Hybrid embeddings + BM25 + metadata
Retrieve	Wide (top-20), rerank
Generate	Cite; allow "I don't know"
Refresh	Same embedding model; re-run retrieval evals

RETRIEVAL BY DATA SHAPE

Data / query	Strategy
Docs, concepts	Hybrid + rerank
Part numbers, codes, IDs	BM25 + exact-match filters
Aggregates ("sales by region")	Read-only query tool
Live state (order, balance)	Tool call to system of record
Scoped by date / permission	Metadata filters
Multi-hop	Agentic iterative search

PROTOCOLS AND CONTEXT STRATEGY

Mechanism	Choose when
API / CLI	One consumer, deterministic call in code
MCP	One integration, many Claude clients; stdio local, Streamable HTTP + OAuth remote
Agent-to-agent	Independent (often cross-org) agents with own tools, state, policy
Monolithic context	Small, static, always needed: cache it
Progressive discovery	Large catalog, sparse use: tool search, Skills, retrieval

API MCP connector: tool calls only, public remote servers, not ZDR-eligible.

OBSERVABILITY AND RELIABILITY

- Log: request ID, model, prompt version, tokens incl. cache, latency, `stop_reason`, tool calls + args, chunk IDs. Redact PII. One trace ID across agents
- `stop_reason: "refusal"` arrives as **HTTP 200**: instrument separately, set fallback per request
- 429: honor `retry-after`, ramp gradually · spend-cap 429: no `retry-after`, alert · 529 overloaded: backoff/fallback
- Patterns: retries · fallbacks · circuit breakers

ACCURACY-LATENCY AND DELIVERY ROUTES

- Levers: caching (TTFT) · fewer reranked chunks · lower effort · faster model on easy routes · parallel tools · streaming (perceived only). **Justify with eval numbers**
- **Claude API:** full features, `inference_geo: "us"` · **Platform on AWS** · **Bedrock** (GovCloud FedRAMP High; some features missing) · **Vertex AI** (FedRAMP High) · **Foundry**
- Procurement, residency, FedRAMP rule routes out **before** latency or cost

D3 TRAPS

Logging instead of removing tools · "only access the user's records" in the prompt · vectors for IDs, aggregates, or live state · MCP for a single fixed call · final-answer-only logs · retrying a spend-cap 429 forever · streaming as an SLA fix.

D4 Evaluation, Testing & Optimization

16% · ~10 items · Lesson 04

OBJECTIVES

1. Define metrics (accuracy, latency, cost, safety, security)
2. Design datasets and mixed-method test frameworks
3. A/B testing and iteration
4. Diagnose prompt failure, hallucination, model mismatch
5. Optimize tokens, latency, cost-performance
6. Monitor with logging and observability

EVALS ARE THE GATE

Grader	Use for	Rule
Code	Labels, schema, exact match, tests	Fastest, most reliable
LLM judge	Faithfulness, tone, rubric	Rubric · reason then score · validate vs. humans · different prompt/model
Human	Calibrate judge, spot checks	Gold; avoid at scale

- Criteria: specific, measurable, achievable, relevant. Volume over hand-graded perfection. Held-out set
- Agents: start with 20–50 real failures. **pass^k**: 75% per run × 3 runs ≈ 42%
- Stages: offline → CI regression → online sampled judging

DIAGNOSE FROM WHAT CHANGED

Trigger	Look at
After document refresh	Retrieval / indexing
Fabricated specifics, retrieval fine	Hallucination: ground, cite, "I don't know"
Rule ignored on some inputs	Prompt: examples, structured outputs
Smaller model, one category fails	Model mismatch: route that category
Newer model, format drifts	Prompt tuned to old model: re-run evals
Cost spike after deploy	Cache hit rate (prefix changed)

A/B TESTING

Hypothesis · one variable · primary + guardrail metrics · sample size up front · report effect size, don't overclaim. Offline evals before online tests. +2 points on 60 samples is noise.

OPTIMIZE COST PER COMPLETED TASK

Caching (input + TTFT) · **Batch API 50%** (async, ≤ 24 h, 100k requests / 256 MB, not ZDR) · **routing** · **effort** · **output length** (output = 5× input price) · streaming = perceived latency only. Include retries and rework.

D5 Governance, Safety & Risk Management

14% · ~9 items · Lesson 05

OBJECTIVES

1. Implement guardrails and safety controls
2. Identify risks, limitations, failure modes
3. Apply human-in-the-loop validation
4. Compliance (GDPR, HIPAA, FedRAMP)
5. Ethics: bias, fairness, transparency

THE SAFETY STACK (FAILS CLOSED)

Layer	Control	Type
Input	Injection/jailbreak screen (fast Haiku), PII redaction	Model + rules
System prompt	Scope, "say I don't know"	**Guidance only**
Tool calls	Least privilege, per-user authz, parameter limits, approvals	**Deterministic**
Output	Schema, PII leak, grounding, policy	Both
Audit	Inputs, tool calls, outputs, reviews	Deterministic

Training reduces harm; the app enforces your rules. Indirect injection: untrusted content in `tool_result`, screen tool outputs, limit what a hijacked agent can do ("no browser agent is immune").

HUMAN-IN-THE-LOOP ROUTING

	Reversible, low cost	Irreversible / high cost
High conf.	Automate + sample	Approval first
Low conf.	Review queue	Human decides

Calibrate on labeled data, not self-reported confidence. Usage Policy high-risk areas need qualified human review + AI disclosure (expanded to 11 areas from Nov 12, 2026; "wholly favorable" decisions exempt).

COMPLIANCE: OBLIGATION → CONTROL → OWNER → EVIDENCE

- **GDPR:** minimize before the call, residency/transfer + DPA, retention limits, human review of legal-effect decisions
- **HIPAA:** BAA before PHI; HIPAA readiness is permanent; Managed Agents and Claude Code not covered; no PHI in schemas
- **FedRAMP High:** Bedrock in AWS GovCloud (also IL4/5) or Vertex AI
- **ZDR not for:** Batch, Files API, Skills, code execution, MCP connector, Managed Agents

FAILURE MODES AND FAIRNESS

- Hallucination · prompt injection · excessive agency · **over-refusal** (measure false-refusal rate) · data leakage · sycophancy · cascading agent errors
- Refusals return HTTP 200 with `stop_reason: "refusal"`
- Fairness: proxies survive column deletion. **Measure outcomes by group**
- Transparency for users, regulators, and your debugging team

D4 + D5 TRAPS

Model swap without evals · unvalidated judge · winner declared on a tiny sample · per-token cost thinking · Batch for live users · rule in the prompt only · fail open when a screen is down · self-rated confidence as router · "fair because we removed gender."

D6 Stakeholder Communication & Lifecycle

14% · ~9 items · Lesson 06

OBJECTIVES

1. Structured discovery and requirements
2. Communicate decisions and trade-offs
3. Feedback loops and expectations (incl. SLAs)
4. Document architectures; implementation guidance
5. Lifecycle: discovery, design, handoff, monitoring, iteration

DISCOVERY

Process + baseline, volume, latency tolerance, cost of a wrong answer, reversibility, data location and sensitivity, regulatory and procurement constraints, the metric. **Write assumptions down.** Never open with technology.

APPROVAL

Each option: **cost · risk · reversal**, in business terms. A recommendation and what would change your mind. Record it in an **ADR**.

HANDOFF

Diagrams, ADRs, prompt/config registry, **eval suite + thresholds**, **runbooks** (symptom → action → owner), named owners. Test: the team ships and rolls back without you.

SLAS AND LOOPS

- Measured: p95 latency, quality floor on sampled review, availability, cost ceiling
- Fallback when degraded; breach → owner + deadline
- **Iterate** on local failures; **re-architect** on structural ones
- Never promise "always correct"

LIFECYCLE

- Deprecation: ≥ **60 days'** notice; Bedrock/Google set own dates → eval successor, re-baseline cost/latency, migrate before retirement
- Pilot: business-critical but low risk; agree **graduation criteria** (performance, operations, risk)
- "Must run in our AWS" → Bedrock first

D7 Developer Productivity & Operational Enablement

7% · ~4 items · Lesson 07

OBJECTIVES

1. Configure Claude tools and environments for teams (e.g., Claude Code)
2. Improve developer workflows with AI tooling
3. Support debugging and operational issue resolution

WHICH LAYER?

Scope	Put here
Managed (no opt-out)	Deny rules, MCP allowlist, marketplace limits, no bypass mode, telemetry
Project (committed)	`CLAUDE.md`, `.claude/settings.json`, hooks, `.mcp.json`, skills
Local (gitignored)	`settings.local.json`, `CLAUDE.local.md`
User	Personal preferences

Precedence: managed → CLI → local → project → user. Permissions: **deny** → **ask** → **allow**.

GUIDANCE VS. ENFORCEMENT

- `CLAUDE.md` = guidance
- Permissions, **hooks** (exit 2 blocks), managed settings = enforcement
- CI: `claude -p`, JSON output, scoped tools, GitHub Action; humans approve; independent reviewer
- Rollout: pilot → shared config → champions → wider; Skills via plugins; spend limits + OTel
- Runbooks map symptoms to architecture causes

DAY Exam-day strategy

Lesson 08

CUE → ANSWER

"only ever need"	Remove tools
"must always", "never"	Code, hooks, permissions, schema
"same prompt every request"	Caching, static first
"overnight", "not user-facing"	Batch API
"after a document refresh"	Retrieval / indexing
"steps are known"	Workflow
"live status", "balance"	Tool call, not RAG
"across organizations"	Agent-to-agent
"every developer, no opt-out"	Managed settings

PROCESS

1. Read the last sentence first
2. Find the trigger or constraint
3. Predict, then read options
4. Label each wrong option with a pattern
5. Pace: item 21 @ 40 min, 42 @ 80, 63 @ 110
6. Flag > 2.5 min; never leave blanks
7. Multiple-response: each option true/false; assume all-or-nothing

Practice: **questions/quiz.html** (60 items) · **anki/ccar-p-flashcards.apkg** (60 cards) · exercises 00–07.